

# **DIGITAL ELECTRONICS**

## **ET 8301**

**Monday, November 13, 2023**

**Mr. Cuthbert J. Karawa**

**Lecture 04**

**BOOLEAN ALGEBRA**

**College of Information and Communication Technology (CoICT)**

**Department Of Electronics And Telecommunications Engineering.**

# Boolean Algebra

- In Boolean algebra, a **variable** is a symbol used to represent an action, a condition, or data. A single variable can only have a value of **1** or **0**.
- Operations in Boolean algebra include **AND**, **OR** and **NOT**.

## Why we need it??

- Algebra is needed to express the output in terms of the input when construct a network of gates

# Boolean Algebra

## ➤ **Boolean Addition.**

- Addition is equivalent to the **OR** operation. The sum term is **1** if one or more of the literals are **1**. The sum term is zero only if each literal is **0**.

## ➤ **Boolean Multiplication**

- In Boolean algebra, multiplication is equivalent to the **AND** operation. The product of literals forms a product term. The product term will be **1** only if all of the literals are **1**.

# Boolean Algebra

## ➤ Boolean NOT.

- The **complement** represents the inverse of a variable and is indicated with an overbar.
- Thus, the complement of **A** is  **$\overline{A}$**

# Rules and Laws of Boolean Algebra

## Commutative Law.

- For **addition**, the commutative law states in terms of the result, the order in which variables are ORed makes no difference.

$$A + B = B + A$$

- For **multiplication**, the commutative law states in terms of the result, the order in which variables are ANDed makes no difference.

$$A . B = B . A$$

# Rules and Laws of Boolean Algebra

## Associative law.

- For **addition**: The associative laws states that when **ORing** more than two variables, the result is the same regardless of the grouping of the variables.

$$A + (B + C) = (A + B) + C$$

- For **multiplication**, the associative law states when **ANDing** more than two variables, the result is the same regardless of the grouping of the variables

$$A(BC) = (AB)C$$

# Rules and Laws of Boolean Algebra

## Distributive Law.

- This law states that **ORing** two or more variables and then **ANDing** the result with a single variable is equivalent to **ANDing** the single variable with each of the two or more variables and then **ORing** the products.

$$AB + AC = A(B + C)$$

and

$$A + (BC) = (A + B) \cdot (A + C)$$

# Rules and Laws of Boolean Algebra

## Distributive Law.

- The **distributive law** is the factoring law. A common variable can be factored from an expression just as in ordinary algebra. That is.

$$AB + AC = A(B + C)$$

and

$$(A + C)(A + B) = A + (BC)$$



# Rules and Laws of Boolean Algebra

## Rules of Boolean algebra.

- Table below shows 13 basic rules that are useful in manipulating and simplifying Boolean expressions.

1.  $A + 0 = A$

2.  $A + 1 = 1$

3.  $A \cdot 0 = 0$

4.  $A \cdot 1 = A$

5.  $A + A = A$

6.  $A + \bar{A} = 1$

7.  $A \cdot A = A$

8.  $A \cdot A = 0$

9.  $\bar{\bar{A}} = A$

10.  $A + AB = A$

11.  $A + \bar{A}B = A + B$

12.  $(A + B)(A + C) = (A + BC)$

13.  $AB + \bar{A}C = (A + C)(\bar{A} + B)$

# Rules and Laws of Boolean Algebra

## Duality.

- Given any logic expression, its dual is formed by replacing all (  $+$  ) with (  $\cdot$  ) and vice versa and replacing all **0s** with **1s** and vice versa.

# Rules of Boolean Algebra

## ➤ Duality Table.

|   | Theorem                                | Its duality                            |
|---|----------------------------------------|----------------------------------------|
| 1 | $a + 0 = a$                            | $a.1 = a$                              |
| 2 | $a + 1 = 1$                            | $a.0 = 0$                              |
| 3 | $a + a = a$                            | $a.a = a$                              |
| 4 | $a + \bar{a} = 1$                      | $a.\bar{a} = 0$                        |
| 5 | $a + ab = a$                           | $a(a + b) = a$                         |
| 6 | $ab + a\bar{b} = a$                    | $(a + b)(a + \bar{b}) = a$             |
| 7 | $a + \bar{a}b = a + b$                 | $a(\bar{a} + b) = ab$                  |
| 8 | $a + bc = (a + b)(a + c)$              | $a(b + c) = ab + ac$                   |
| 9 | $ab + \bar{a}c = (a + c)(\bar{a} + b)$ | $(a + b)(\bar{a} + c) = ac + \bar{a}b$ |

# Rules and Laws of Boolean Algebra

## De Morgan's Theorem.

- **De Morgan's 1<sup>st</sup>** Theorem states that the complement of a product of variables is equal to the sum of the complemented variables.

$$\overline{AB} = \bar{A} + \bar{B}$$

- **De Morgan's 2<sup>nd</sup>** Theorem states that The complement of a sum of variables is equal to the product of the complemented variables.

$$\overline{A + B} = \bar{A} \cdot \bar{B}$$

# Boolean formulas and functions

- A Boolean function is an expression formed with binary variables, the two binary operators **AND** and **OR**, one unary operator **NOT**, parentheses and equal sign.
- Boolean functions can also be represented by **truth tables**.
- A Boolean function from an algebraic expression can be realized to a logic diagram composed of logic gates.
- **Example**

$$f(a, b, c) = a\bar{b}c + \bar{a}b + \bar{b}\bar{c}$$

$$f(x, y, z) = (xy + y\bar{z} + \bar{x}\bar{y}z)$$

# Boolean formulas and functions

- **Example:** Examine whether the following expressions are equivalent.

$$f_1 = \overline{(a + b)} \overline{(\bar{a} + \bar{b})}, f_2 = \bar{a}\bar{b}(ab)$$

$$a + \bar{a}b \text{ and } a + b$$

# Sum of Products Form (SOP)

- SOP form the Boolean expression is written either as **single product term** or as a **sum of product terms**.
- A product term consists of either a single literal or a product of more than one literals.
- **Example:**

$$\bar{b} + abc + a\bar{c}$$

# Product of Sum (POS)

- In POS form the Boolean expression is written either as a single sum or as a product of sum terms.
- A sum term consists of either a single literal or a sum of more than one literals.
- Example:

$$\bar{x}(y + \bar{z})(x + y + \bar{z})$$



# Canonical form

- An expanded form of Boolean expression, where each term contains all Boolean variables in their true or complemented form, is also known as the **canonical form** of the expression.
- Boolean expression can be written from truth tables.
- Two canonical forms which known as **minterm** and **maxterm** canonical form can be obtained directly from truth table.

# Canonical form

## Minterm Canonical Form.

- Minterms are product terms where all variables appear once either in complemented or uncomplemented form.
- Expression constructed by minterms are said to be in minterm canonical form or standard sum of product.
- m-Notation: is used to simplify writing functions in minterm canonical forms.
- Minterms are represented as  $m_i$  where  $i$  stands for the row number for which the function evaluates to one.

# Canonical form

## Minterm Canonical Form.

- Example: Write the function represented in the table as an expression in minterm canonical form as well as in m-notation

| a | b | c | f | terms                   | $m_i$ |
|---|---|---|---|-------------------------|-------|
| 0 | 0 | 0 | 0 | $\bar{a}\bar{b}\bar{c}$ | $m_0$ |
| 0 | 0 | 1 | 0 | $\bar{a}\bar{b}c$       | $m_1$ |
| 0 | 1 | 0 | 1 | $\bar{a}b\bar{c}$       | $m_2$ |
| 0 | 1 | 1 | 0 | $\bar{a}bc$             | $m_3$ |
| 1 | 0 | 0 | 0 | $a\bar{b}\bar{c}$       | $m_4$ |
| 1 | 0 | 1 | 1 | $a\bar{b}c$             | $m_5$ |
| 1 | 1 | 0 | 0 | $ab\bar{c}$             | $m_6$ |
| 1 | 1 | 1 | 1 | $abc$                   | $m_7$ |

$$f = \bar{a}b\bar{c} + a\bar{b}c + abc$$

$$f = m_2 + m_5 + m_7$$

$$f = \sum m(2,5,7)$$

# Canonical form

## Maxterm Canonical Form.

- Maxterms are sum terms where all variables appear once either in complemented or uncomplemented forms.
- Expression constructed by maxterms are said to be in **maxterm canonical form** or **standard product of sum**.
- While expressing functions in maxterm canonical form we look out rows for which the function evaluates to 0.
- The **M-notation** is used to simplify writing in maxterm canonical form.
- Maxterms are represented as  $M_i$  where  $i$  stands for row number for which the function evaluates to **0**.

# Canonical form

## Example.

- Write function represented in the table below as an expression in maxterm canonical form as in M-notation

| $x$ | $y$ | $z$ | $f$ | $terms$                       | $M_i$ |
|-----|-----|-----|-----|-------------------------------|-------|
| 0   | 0   | 0   | 1   | $a + b + c$                   | $M_0$ |
| 0   | 0   | 1   | 1   | $a + b + \bar{c}$             | $M_1$ |
| 0   | 1   | 0   | 0   | $a + \bar{b} + c$             | $M_2$ |
| 0   | 1   | 1   | 1   | $a + \bar{b} + \bar{c}$       | $M_3$ |
| 1   | 0   | 0   | 1   | $\bar{a} + b + c$             | $M_4$ |
| 1   | 0   | 1   | 0   | $\bar{a} + b + \bar{c}$       | $M_5$ |
| 1   | 1   | 0   | 0   | $\bar{a} + \bar{b} + c$       | $M_6$ |
| 1   | 1   | 1   | 1   | $\bar{a} + \bar{b} + \bar{c}$ | $M_7$ |

$$f(x, y, z) = (x + \bar{y} + z)(\bar{x} + y + \bar{z})(\bar{x} + \bar{y} + z)$$

$$f(x, y, z) = M_2 \cdot M_5 \cdot M_6$$

$$f(x, y, z) = \prod M(2,5,6)$$

# Combinational logic circuits

- Combinational circuit is an interconnection of gates where the **logic level of the outputs at any time depends only on the inputs at that time.**
- A logic circuit with no interconnection between output terminals and input terminals are called **combinational circuits.**
- Circuits are used in combinatorial logic to **carry out operations** such as **data manipulation** and **selection, coding, decoding** and **error detection.**
- Among the basic components listed as being necessary to implement them, there are **multiplexers, demultiplexers, encoders, decoders** and **shifters.**

# Combinational logic circuits

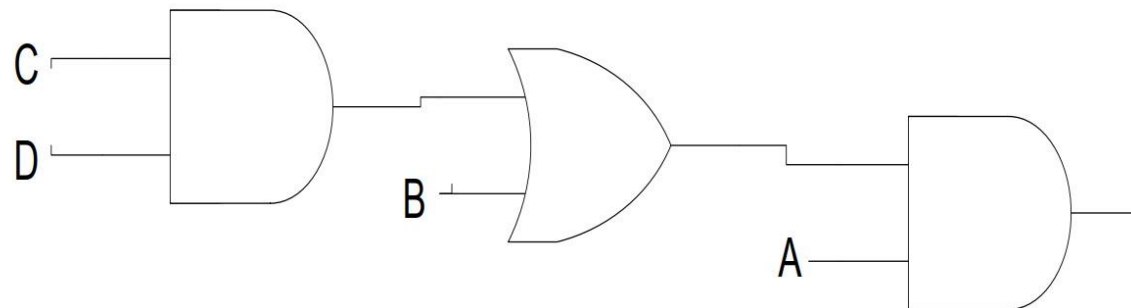
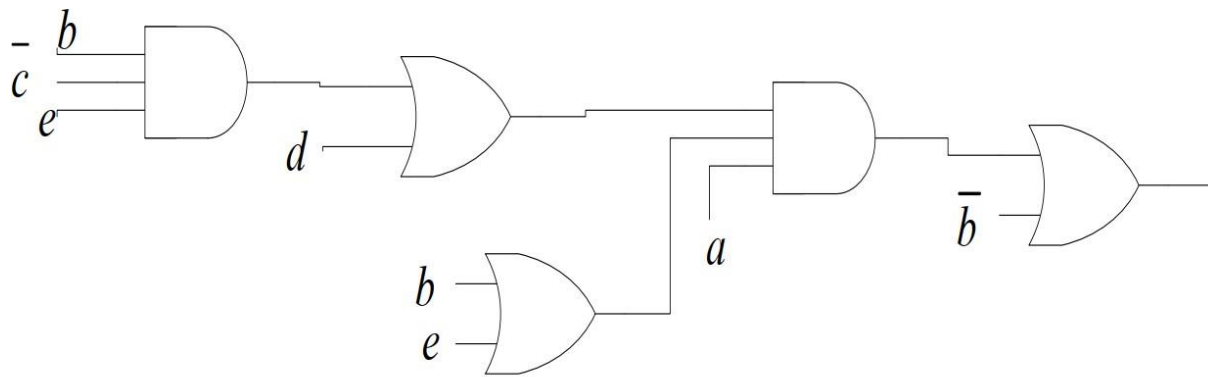
## **Analysis of combinational circuits.**

- During analysis it assumed that the inputs are at one of either two possible values.
- While analyzing combinational circuits start with the gate at which the input are connected and continue until the output is reached.

# Combinational logic circuits

## Analysis of combinational circuits.

- Example: Analyze the circuits below by writing its Boolean expression.





# Combinational logic circuits

## **Constructing a truth table for a logic circuit.**

- Once the Boolean expression for a given circuit has been determined a truth table that show the output for all possible values of the input variables can be developed.

## **Synthesis of combinational Circuits**

- This involve the design of combinational circuit resulting in its schematic diagram or gate diagram.

# Combinational logic circuits

## Example.

➤ Synthesize the following Boolean expressions.

1.  $f(a, b, c, d) = \bar{c}(a + d) + \bar{b}c$

2.  $f(a, b, c, d) = \bar{a}(b + c)(\bar{b} + d) + b\bar{d}$

# Combinational logic circuits

## Combinational Logic Design.

- Logic design involves the generation of the schematic diagram given the specifications.
- The statement is translated into truth table and implement using two level logic.

## Examples

- Design a system which accepts four input a, b, c and d and outputs a 1 whenever the number of 1s in the input exceeds 2.
- Design a system which accepts four inputs w, x, y and z outputs 1 whenever the least significant input or the most significant inputs goes high.

# Combinational logic circuits

## Realization using NAND gate.

- If given an expression in SOP form the final expression is an OR operation and the expression can be implemented right away.
- If given expression is in POS form where the final operation is a AND operation we need to apply the De Morgan's theorem and implement the complement of the function and finally invert using a NAND gate configured as an inverter.

# Combinational logic circuits

## Realization using NAND gate.

### Examples

- Realize the following Boolean expression using only NAND gates

a.  $Y = \bar{a}b + ac$

b.  $Y = (a + c)(\bar{b} + \bar{d})(\bar{a} + \bar{b} + \bar{c})$

# Combinational logic circuits

## Realization using NOR gate

- If given an expression in POS form the final expression is an AND operation and the expression can be implemented right away.
- If given expression is in SOP form we need to apply the De Morgan's theorem and implement the complement of the function and finally place a NOR gate configured as an inverter.

# Combinational logic circuits

## Realization using NOR gate

### Example.

- Implement the following expressions using only NOR gates.

*a.*  $Y = \bar{a}b + ac$

*b.*  $Y = (a + c)(\bar{b} + \bar{d})(\bar{a} + \bar{b} + \bar{c})$

# Tutorials

- Prove the following identities.

*a.*  $ab + a\bar{b} + \bar{a}\bar{b} = a + \bar{b}$

*b.*  $(a + b)(\bar{a}\bar{c} + c)(\overline{\bar{b} + ac}) = \bar{a}b$

- By induction prove that

$$ab + bc + \bar{a}c = ab + \bar{a}c$$

- Establish equivalence of expression

and  $f_2 = (a + c + \underline{d})(a + c + \bar{\underline{d}})(a + \bar{c} + d)(a + \bar{b})$

$$f_2 = a + bc\bar{d}$$



# Tutorials

- For the truth table shown below Write the minterm and maxterm canonical form using m-notation and M-notation respectively of Boolean expression represented.

| a | b | c | f |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

# Tutorials

- Using theorem and Boolean Algebra simplify the following

*a.*  $\bar{a}\bar{b}\bar{c}d + a\bar{b}\bar{c}d + bd + bcd\bar{d}$

*b.*  $(a + b + \bar{c} + d)(a + \bar{b} + \bar{c} + d)(a + \bar{b} + \bar{c} + \bar{d})(\bar{a} + \bar{b} + \bar{c} + d)(\bar{a} + \bar{b} + \bar{c} + \bar{d})$

- Convert the following to its minterm canonical form.

*a.*  $(a + \bar{b})(a + c)$

*b.*  $ab + \bar{c}$

- Convert the following to its maxterm canonical form

*a.*  $a + \bar{a}\bar{c}(b + c)$

# Tutorials

- Write the expression in algebraic form for the function.  
$$f(w, x, y, z) = \sum m(4, 8, 10, 14)$$
- Design a circuit to square a two bit number assuming.
- Design a circuit to generate the even parity bit for a system with four input a, b, c and d assuming double rail logic. Hint. The output should be such that there must be an even number of 1s with the inputs and outputs put together for each row.
- Write step of realizing Boolean expression using NAND and NOR by graphical approach. Then use graphical approach to realize the following function using multilevel NAND-NAND network and NOR-NOR network.

$$f = \bar{a}b + b(c + d) + e\bar{f}(\bar{b} + \bar{d})$$





**Next Lecture**  
**Design of**  
**combinational logic**  
**circuits**